

SmoothRL: Online Reinforcement Learning During Asynchronous Execution

Atribot Team

`research@atribot.com`

Project Page: <https://www.atribot.com/research/SmoothRL>

Author List in Contributions

Abstract

Deploying robot policies in the physical world requires simultaneously satisfying two fundamental desiderata: reliability, as reflected by high task success rates, and smooth real-time execution. However, deploying state-of-the-art generalist models, including Vision-Language-Action (VLA) models and World-Action Models (WAMs), presents significant challenges on both fronts. Achieving the precision and robustness required for real-world deployment necessitates sample-efficient online reinforcement learning (RL) to adapt pretrained models for reliable deployment. Meanwhile, the increasing scale of robot foundation models has led to correspondingly higher inference latency. To satisfy real-time constraints under high latency, modern systems adopt asynchronous inference together with action chunking, overlapping policy computation with chunk execution to hide latency and enable smooth control. Despite their complementary roles, integrating asynchronous execution with gradient-based online RL remains underexplored. We present SmoothRL, an online RL framework that fine-tunes a pretrained policy within an asynchronous inference loop. SmoothRL follows a value-gradient paradigm, directly updating the policy parameters using gradients of the action-value function with respect to the policy actions. To enable correct optimization under asynchronous execution, SmoothRL explicitly models the asynchronous inference process during training. Specifically, each generated action chunk is partitioned by frame index into three regions: a *committed region*, which consists of actions committed by the previous inference cycle; an *execution region*, which contains newly generated actions that are executed by the robot; and a *discarded region*, which contains actions that will be superseded by the next inference cycle. Gradients are propagated only through the execution region, ensuring that policy optimization is aligned with the trajectory distribution induced by asynchronous execution. We evaluate SmoothRL on real-world robotic tasks requiring high precision, as well as highly dynamic tasks that necessitate asynchronous execution.

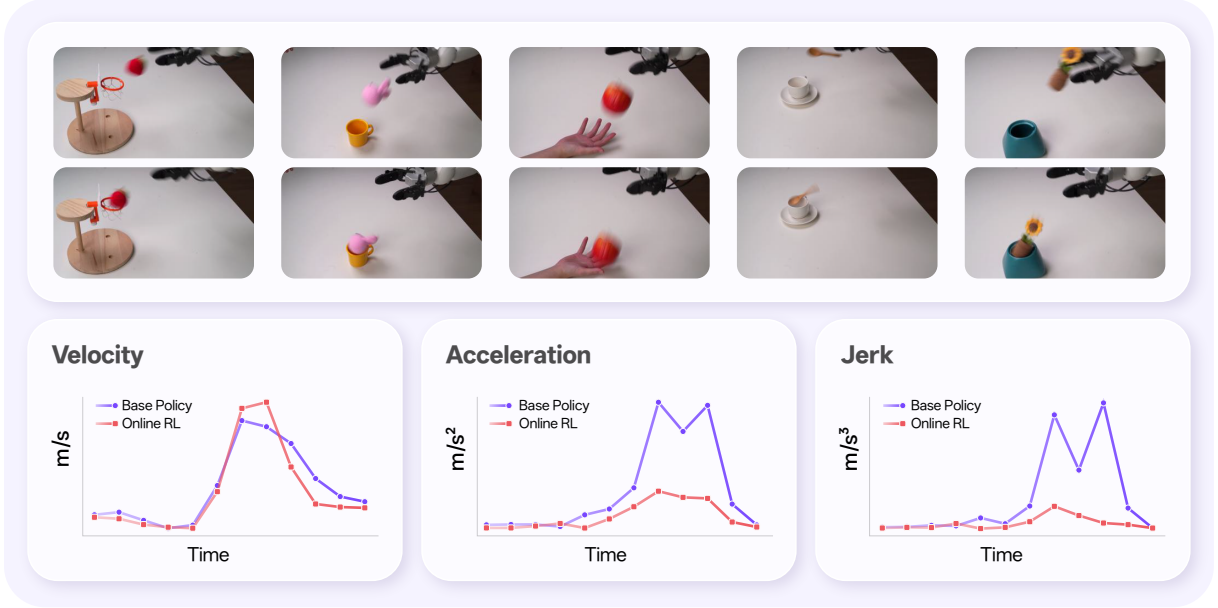


Figure 1 | **Top:** SmoothRL enables online RL fine-tuning of pretrained robot policies under asynchronous inference, supporting highly dexterous and dynamic tasks such as throwing an object at a target. **Bottom:** SmoothRL produces smoother motion, reducing acceleration and jerk by 52% and 47%, respectively. Values are RMS measurements of the right end-effector’s XYZ motion over each action chunk during a real autonomous throwing rollout.

1. Introduction

Recent advances in Vision-Language-Action (VLA) models and World-Action Models (WAMs) have demonstrated remarkable progress toward endowing robots with general manipulation capabilities [40, 42–49]. However, deploying such generalist policies in the physical world requires simultaneously addressing two fundamental challenges: **reliability** and **smooth real-time execution**. Achieving the robustness required for diverse real-world tasks such as cutting open sealed packages demands sample-efficient online reinforcement learning (RL). Meanwhile, the increasing scale of robot foundation models has led to correspondingly higher inference latency. Under synchronous execution, this latency lies directly on the control path: the robot must pause at every chunk boundary while waiting for the next forward pass to complete. When success depends on maintaining continuous motion, such pauses can severely degrade both task success and execution efficiency. Modern robot systems therefore adopt action chunking together with asynchronous inference, overlapping policy inference with action execution. Asynchronous execution eliminates these pauses but introduces a discontinuity of its own: each incoming chunk is generated from an earlier observation, so switching to it can produce a discontinuity at the junction. A family of chunk-stitching methods [1–4] therefore studies how to maintain continuity between newly generated actions and those already committed for execution. Although reliability and smooth real-time execution are complementary requirements for real-world deployment, they have largely been studied in isolation. Existing online RL methods typically optimize policies assuming synchronous execution, while deployment almost invariably relies on asynchronous inference. This mismatch causes the policy to be optimized under dynamics that differ from those encountered at test time. We argue that online RL should instead be performed under the same asynchronous execution paradigm used during deployment: **Reinforce in Deployment**.

Pursuing reliability and smooth real-time execution simultaneously reduces the design space to three essential requirements. **Online**: the policy improves from the data it is currently generating, rather than through offline training rounds. **Asynchronous**: inference overlaps execution so that model inference latency does not limit the control frequency. **Value-gradient**: the gradient of the action-value function with respect to the policy actions is backpropagated into the policy parameters, allowing the learning signal to directly optimize the policy. Existing approaches each sacrifice one of these requirements, in the same order. **Offline RL** separates learning from deployment by conducting RL in offline rounds, leaving deployment free to be asynchronous. Estimated advantages influence policy updates through reweighting or conditioning, rather than direct gradient optimization through the value function [29, 30]. **Synchronous Online RL** performs online reinforcement learning by backpropagating gradients from the value function Q into the policy parameters. While effective for adapting pretrained policies [7, 9, 11–13], these methods assume synchronous execution. **Asynchronous Restricted Online RL** considers online adaptation under asynchronous execution. However, policy optimization is confined to a latent steering space rather than the policy parameters themselves, making the achievable improvement ultimately bounded by the decoding capability of the pretrained base policy [31]. **Asynchronous Online RL**, which satisfies all three requirements, is the focus of this work. Under asynchronous execution, each generated action chunk is only partially executed before a newly inferred chunk supersedes its remaining actions. However, policy updates should be driven exclusively by the consequences of the actions the robot actually executes. Unlike the three settings above, asynchronous online RL cannot avoid gradient contamination by design: taking the value-gradient pathway while deploying asynchronously inevitably exposes the policy parameter update to un-executed, superseded actions. The objective itself must therefore eliminate this contamination, rather than relying on the execution schedule.

We present SmoothRL, a general framework for online RL fine-tuning of pretrained robot policies under asynchronous inference, enabling highly dexterous and dynamic tasks, such as throwing an object at a target, as illustrated in Figure 1. The framework follows the value-gradient paradigm, where the action-value function Q updates the policy parameters directly through its gradient with respect to the policy actions. This paradigm admits either fine-tuning the pretrained base policy directly or learning a lightweight policy on top of a frozen one; we adopt the latter as one instantiation, where a frozen base policy generates reference action chunks under a receding horizon and a trainable attached network in the raw action space carries all learnable parameters. Under asynchronous execution, each generated action chunk is partitioned according to its execution status into three regions: a *committed region*, containing actions already issued during the previous inference cycle; an *execution region*, containing newly generated actions that are actually executed by the robot; and a *discarded region*, containing newly generated actions that are superseded by the next inference cycle before execution. The boundaries between these regions are determined by two distinct forward passes and are unknown at chunk generation time, unless the inference loop is constrained to a fixed latency budget. Moreover, because inference and execution overlap, a chunk begins execution only after the preceding in-flight inference has completed, making each decision inherently concurrent [35]: the action chunk currently in flight is part of the state against which the next chunk is generated. The two contributions below formalize the resulting implications for training:

- **Gradient truncation to the execution region.** Holding the asynchronous loop to a latency budget of n frames fixes both boundaries of the execution window, making the three action-chunk regions a function of frame index alone. The value gradient $\nabla_a Q$ is then computed only with respect to the execution region $a_{[n, 2n)}$, ensuring that policy updates

depend exclusively on actions whose consequences are observed. The critic, however, retains the committed region as part of its input for two reasons: the chunk-skip bootstrap is unbiased only when the critic is conditioned on the action sequence that generated the interval reward, while the action currently in flight provides the state augmentation required to model the concurrent decision process.

- **Asynchronous execution embedded in the training loop.** The asynchronous inference loop - where policy forward passes and action execution overlap in time - runs during training rollouts and not only at deployment. Replay trajectories therefore record the actions the robot actually executed under the same timing and execution schedule the objective is defined against, ensuring the policy is never optimized under dynamics it will not encounter at deployment.

A further benefit of operating in the raw action space is that human interventions lie in the same space as the policy outputs. Consequently, a demonstrated chunk can be used directly, without latent inversion or a separate offline data-collection stage: it serves as a regression target for the policy under a behavioral cloning loss over the execution region and as a transition for the critic under the standard TD backup. We evaluate SmoothRL on a suite of real-world robotic manipulation tasks requiring high precision, as well as highly dynamic tasks that necessitate asynchronous execution.

2. Related Work

We first review mechanisms for asynchronous policy deployment, and then organize existing RL approaches along the three dimensions introduced in §1: whether learning is online or offline, whether deployment is synchronous or asynchronous, and whether value gradients are propagated directly to the policy.

2.1. Asynchronous Policy Deployment

Receding-horizon action chunking has become the standard deployment paradigm for contemporary large-scale generalist robot policies [5, 6, 40, 42, 47, 48]. Since generating each action chunk requires a complete policy forward pass, typically taking tens to hundreds of milliseconds, asynchronous inference is a practical necessity: synchronous per-segment forwards would pin control frequency to an unacceptable rate [36–38]. Consequently, each newly generated chunk is conditioned on a stale observation and is unaware of the actions executed since that observation was captured. As a result, successive chunks generally fail to connect smoothly along the executed trajectory, introducing velocity or acceleration discontinuities at chunk boundaries. One line of work addresses this problem through **consistency-aware chunk-stitching** [1–4]. These methods constrain the already-consumed prefix of each newly generated chunk to match the actions that have already been executed, while leaving the remaining horizon unconstrained. A related approach instead corrects an in-flight chunk using newly acquired observations [39]. Over time, these methods have evolved from inference-time guidance external to the policy toward training-time modeling within the policy, and from hard prefix constraints toward soft interpolation. An orthogonal line of work employs **trajectory blending** [6, 29], which smooths chunk boundaries by post-processing the overlapping predictions of successive chunks using a decaying interpolation weight. Rather than constraining the policy outputs themselves, trajectory blending operates only after multiple chunks have been generated. Although both

approaches eliminate discontinuities, they differ fundamentally in when the executed action is determined: chunk stitching fixes the action before it is emitted by the policy, whereas trajectory blending determines it only after the chunks overlapping that instant become available. This distinction is inconsequential for imitation learning and offline RL, where both families have primarily been studied. However, it becomes decisive for the online RL objective, which requires the action at which the value function Q is evaluated to be exactly the action executed by the robot, while simultaneously preserving a differentiable path from that action to the policy parameters. This requirement raises a new question for asynchronous online RL, which we address in this work.

2.2. Offline RL

Offline RL consumes the learning signal entirely during training, leaving the policy fixed at deployment. Methods such as χ_0 [29] and RECAP [30] support asynchronous deployment: the advantage is estimated by a separate value model and influences the policy only through weighting (AWR [50]) or conditioning (CFG-style guidance [51]), rather than by backpropagating value gradients into the policy parameters. Conversely, CO-RFT [10] performs all chunk-level Q -learning offline (Chunked Cal-QL) but adopts synchronous deployment. The coexistence of both deployment modes within this class shows that, for offline RL, asynchronous execution is merely an inference-scheduling choice and does not alter the learning signal. In contrast, our method closes the actor-critic loop around real-robot execution, where the mismatch between generated and executed actions directly affects the learning signal.

2.3. Synchronous Online RL

This body of work performs online actor-critic updates in the same loop as robot rollout. All methods in this category follow the value-gradient paradigm, in which the action-value function backpropagates gradients directly into the policy parameters. They differ primarily in where reinforcement learning is applied: either on a lightweight module attached to a frozen pretrained policy, or on the entire policy through end-to-end fine-tuning. One line of work freezes the pretrained VLA and learns a chunk-level policy on top. RLT [9], whose architecture our implementation directly adopts, appends a learnable RL token to a frozen VLA and conditions a chunk-level actor on both the resulting representation and the VLA reference action. The actor is optimized with a TD3-style [58] off-policy objective, while a behavior-cloning regularizer constrains learning to local corrections. EXPO-FT [7] similarly attaches a residual editing head to a frozen VLA and optimizes it using a SAC-style [60] actor objective together with Best-of- N selection, placing it within the same value-gradient framework. Other methods retain the frozen-base interface but learn per-step residual policies rather than chunk-level ones [11–13, 27, 28], reducing the RL optimization unit from an action chunk to a single control step. Chunk-level off-policy methods outside the frozen-policy setting [34, 52] nevertheless adopt the same chunk-level optimization unit as our approach. This shared optimization unit motivates our formalization of the **action chunk as the fundamental RL optimization unit** (§3.2). In a per-step formulation, actions carry no frame index within a chunk, making it impossible to express gradients that apply only to a selected sub-segment of that chunk. A second line of work applies reinforcement learning directly to the entire policy. Representative examples include end-to-end token-level optimization for VLAs [15–18], reinforcement learning of flow- and diffusion-based policies [8, 19–22], and consistency-distillation approaches [24, 25]. FQL [23], while operating in an offline-to-online rather than closed-loop real-robot setting,

distills a pretrained flow policy into a one-step policy that directly receives value gradients, a design also explored as a variant of QAM [55]. LWD [26] closes the loop on real-robot execution by performing fleet-scale online updates with a chunk-level critic and propagating $\nabla_a Q$ into a flow-policy action head through an adjoint-matching formulation. Despite their differences, all methods in this category assume synchronous receding-horizon execution, so partial chunk execution never enters their formulation. By contrast, opening the deployment loop to asynchronous inference turns partial execution into a constraint on the value-gradient pathway. Our method addresses this constraint through chunk-role partitioning and gradient truncation, both defined over frame indices within an action chunk. Because the formulation operates directly on raw action chunks rather than a particular policy parameterization, it applies equally to residual-policy adaptation and end-to-end policy fine-tuning.

2.4. Asynchronous Restricted Online RL

Online actor-critic learning requires exact chunk transitions, yet asynchronous execution causes each generated action chunk to be only partially executed, contaminating those transitions. Existing methods avoid this issue by keeping the value gradient, $\nabla_a Q$, away from the raw-action policy parameters. Instead, reinforcement learning is performed in latent space, through value-based action selection, or by using the value function only to rank or verify sampled candidates [53, 54]. Our method instead follows the value-gradient paradigm, embedding asynchronous execution directly into the RL objective as a known optimization constraint. GR-RL [31] performs online learning under asynchronous execution, but places the RL optimization variable in the latent noise space of a frozen diffusion policy [32]. It learns a latent-noise predictor, while a frozen decoder maps the optimized latent back to actions. Optimizing the latent noise therefore selects among actions already represented by the pretrained policy prior. Consequently, partial execution is bounded in its effect rather than eliminated: if value estimation includes un-executed portions of an action chunk, the resulting error may alter which latent candidate is selected, but the frozen decoder ensures that every issued action remains within the action distribution represented by the pretrained prior. This property bounds not only the error but also the attainable improvement, which remains limited by the expressive capacity of the frozen decoder. Moreover, human intervention cannot be incorporated as direct policy regression targets without unreliable latent inversion. In contrast, our method optimizes directly in raw action space, allowing human intervention data to serve simultaneously as supervised regression targets and value-labeled online transitions.

3. Method

Our work addresses one single challenge:

How can online RL fine-tuning be integrated into an asynchronous deployment loop ?

3.1. The Asynchronous Inference Loop

We exclusively consider an asynchronous inference setting to satisfy the smooth real-time execution requirements of real-world deployment, which has become the standard paradigm for contemporary large-scale generalist robot policies. The base policy infers the next action chunk while the robot continues executing the current one. Meanwhile, the control process operates at

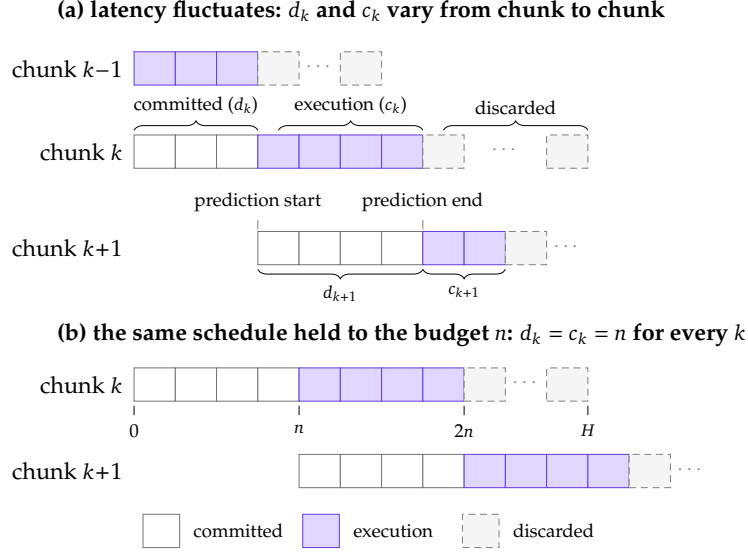


Figure 2 | Chunk role partitioning under asynchronous execution. The committed region reuses actions already issued by the previous inference, the execution region contains the frames executed from the current chunk, and the discarded region contains frames superseded by the next inference. Chunk k 's committed length d_k is determined by the latency of its own inference, whereas its execution length c_k is determined by the latency of the subsequent inference cycle. (a) Under variable inference latency, d_k and c_k fluctuate independently across chunks. (b) The same schedule under a latency budget of n , where $d_k = c_k = n$ for every chunk, yielding a fixed execution window $[n, 2n)$. Section 3.1 details the partition, and Section 3.3 defines the learning objective based on the partition.

the desired control frequency and always issues the latest available action without waiting for inference to complete; once a new chunk becomes available, it immediately takes over execution. This design effectively hides inference latency.

Asynchronous execution introduces a fundamental consequence: **a policy-generated action chunk is only partially executed by the environment**. We therefore partition each action chunk of horizon H into three regions by frame index, according to how the asynchronous schedule uses the generated actions. The **committed region** $[0, d)$ contains the frames that elapse during the inference latency of this chunk. Its length d is determined by the current inference time. By the time the chunk becomes available, these frames have already passed: the previous chunk has supplied the corresponding actions, and the predictions in this region can never affect the environment through the current chunk. The **execution region** $[d, d + c)$ corresponds to the interval during which this chunk is the latest available policy output before the next inference completes. Its length c is determined by the arrival time of the subsequent chunk. Every action in this region is issued to the robot exactly as generated, making it the only part of the chunk that directly determines the resulting environment trajectory. The remaining **discarded region** contains frames after the next chunk becomes available. These actions are superseded by the newly arrived chunk and therefore never reach the environment.

Stabilizing the loop with a latency budget. The boundaries between chunk regions, illustrated in Figure 2 (a), are stochastic in real deployments. Since they are determined by different inference cycles, they fluctuate independently and are unknown when an action chunk is generated. Rather than modeling this stochasticity, we eliminate it through a fixed latency budget. We estimate an upper bound n on the inference time and enforce a scheduled execution

loop: each chunk is requested n control steps before its first frame is due. If inference completes earlier, the system waits, ensuring that chunk handover always occurs at the predefined time. Consequently, both region lengths are fixed by the same constant, $d = c = n$. For any horizon $H \geq 2n$, the committed region is therefore $[0, n)$, the execution region is $[n, 2n)$, and the discarded region is $[2n, H)$ (Figure 2 (b)). The slack between the actual inference latency and the budget n is converted into waiting time rather than used to exploit fresher observations. In exchange, every chunk is executed over an identical window determined solely by its frame indices.

3.2. Gradient-based RL in Continuous Action Space

This subsection reviews the general formulation of continuous-action reinforcement learning used throughout the paper. Given a Markov decision process (MDP), a policy π seeks to maximize the expected discounted return. Value-based methods define the action-value function $Q^\pi(s, a)$, which satisfies the Bellman equation:

$$Q^\pi(s, a) = \mathbb{E}_{r, s' \sim p} [r + \gamma Q^\pi(s', \pi(s'))], \quad (1)$$

where γ is the discount factor and s' is the successor state reached after executing action a in state s . Off-policy actor-critic methods alternate between two updates. The **critic** learns the action-value function Q by fitting the temporal-difference (TD) target given by the right-hand side of the Bellman equation using transitions sampled from a **replay buffer**. The **actor** then updates the policy parameters to increase the predicted action value, i.e., to maximize Q .

In continuous action spaces, maximizing the action value can be realized through two distinct routes. One route improves actions through candidate selection, advantage weighting, or latent-space guidance, without backpropagating $\nabla_a Q$ to the policy parameters that generate the executed actions. Our framework instead follows the value-gradient route: the gradient of the action-value function with respect to the action, $\nabla_a Q$, is backpropagated through the deterministic policy $a = \pi_\theta(s)$ to the policy parameters θ ,

$$\nabla_\theta J = \mathbb{E} [\nabla_a Q(s, a)|_{a=\pi_\theta(s)} \cdot \nabla_\theta \pi_\theta(s)], \quad (2)$$

so that Q directly updates the policy through action-space gradients.

Action Chunk as the Atomic Unit of RL. We formulate reinforcement learning over action chunks, treating each chunk as an atomic action in the underlying MDP. For a chunk spanning H frames, the transition is represented as $(s_t, a_{t:t+H}, r_t, s_{t+H})$, where $a_{t:t+H}$ is the action chunk generated by the policy at state s_t , r_t is the discounted return accumulated during execution of the chunk, and s_{t+H} is the observation after the chunk has been fully executed. When indexing frames relative to the start of a chunk rather than to the episode - as in the chunk regions defined in §3.1 - we denote the chunk by $a_{[0, H)}$ and a sub-segment by $a_{[i, j)}$. The corresponding action-value function is written $Q(s_t, a_{t:t+H})$, and its temporal-difference update follows the chunk-skip Bellman backup:

$$Q(s_t, a_{t:t+H}) \leftarrow r_t + \gamma^H Q(s_{t+H}, \pi(s_{t+H})). \quad (3)$$

This backup remains unbiased because the critic is conditioned on the action sequence that generated the reward over the corresponding interval. The same formulation has since been adopted by recent chunk-level RL methods (§2.3), where the action span on which the critic is conditioned need not coincide with the span for which the policy is optimized [14]. Our framework likewise adopts this formulation, with the modification required to account for asynchronous execution.

3.3. Chunk-Level RL under Asynchronous Deployment

Under asynchronous deployment, directly concatenating the two components above - allowing $\nabla_a Q$ to backpropagate through the entire chunk - would cause policy updates to depend on generated actions that never reached the environment. Instead, we explicitly treat “which segment is actually executed” as a known constraint imposed by the deployment process.

The regions introduced in §3.1 were originally defined solely by the execution schedule; here, they are assigned distinct training roles. From this point onward, the span over which the objective is defined is restricted to the first $2n$ frames of the chunk, with the committed and execution regions of §3.1 each occupying one budget interval; the horizon H the base policy emits may be longer. We denote by $\tilde{a}_{[0,n]}$ the actions actually executed by the robot in the committed region of the current chunk. These actions were issued by the preceding chunk and are treated as fixed, regardless of how they were generated. The objective defined in this subsection is therefore:

$$\max_{\theta} \mathbb{E} \left[Q(s, \tilde{a}_{[0,n]}, \pi_{\theta}(s)_{[n,2n]}) \right] \quad \text{s.t.} \quad (\tilde{a}_{[0,n]}, \pi_{\theta}(s)_{[n,2n]}) \in \mathcal{E} \quad (4)$$

where $\mathcal{E}$ denotes the set of chunks over $[0, 2n]$ that are physically executable by the robot. Importantly, admissibility is a property of the full chunk span, not merely the execution region: a chunk is included only when its execution is valid within both regions and remains consistent across the boundary at frame n . The discarded region $[2n, H)$, which falls outside the effective execution horizon, is therefore excluded entirely from the objective above. The chunk-skip backup of Eq. (3) is restricted to the same span: the interval it spans is $[0, 2n]$ rather than the full horizon H , so the reward r_t accumulates over those $2n$ frames, the bootstrap state is s_{t+2n} , and the discount exponent is $2n$. This modification preserves the required unbiased backup: the critic is conditioned on the action sequence that actually generated the reward over the interval, namely $\tilde{a}_{[0,n]}$ followed by $a_{[n,2n]}$.

Value gradient truncation. From the policy’s perspective, the committed region induces no valid environmental feedback for policy updates: as described in §3.1, these actions are overwritten by those issued from the preceding chunk and therefore do not form a valid gradient path for the current policy. Accordingly, the value gradient is truncated to the execution region: $\nabla_a Q$ is computed only with respect to $a_{[n,2n]}$. This truncation establishes a crucial identity: the segment receiving the value gradient is simultaneously the policy output and the action actually executed by the environment, making the resulting gradient well-defined with respect to the policy parameters. Importantly, the truncation applies only to the gradient path, while the critic continues to operate on the full action chunk.

Critic over the full action span. Asynchronous execution turns this setting into a concurrent decision problem. The execution region does not begin acting from s_t , but from s_{t+n} , with the intervening frames being governed by the chunk already in flight. A critic conditioned only on $(s_t, a_{[n,2n]})$ would therefore marginalize over the unknown distribution of in-flight chunks present in the replay buffer, which, in the off-policy setting, reflects behaviors induced by older policies. A concurrent decision process requires augmenting the state with two quantities [35]: the action currently in flight and the remaining time until its completion. In our setting, the latency budget fixes the latter to the constant n , while chunk-stitching consistency makes the

former directly available at generation time: for each newly generated chunk, its committed region exactly corresponds to the actions already issued by the preceding chunk. Recording these issued actions therefore provides the required state augmentation both freely and exactly. The same latency budget that determines this augmentation also fixes the timescale of the bootstrap chain, which we analyze further in §5.

Smoothness. The executable-set constraint restricts not the frames over which the policy can act, but the set of chunks over which the optimization is performed. The actor also conditions on the committed region, but for a different reason than the critic: since the committed and execution regions are connected at frame index n , a candidate chunk must be selected with awareness of the trajectory it must continue from. Smoothness inherited from the pretrained policy is provided by the demonstrations, rather than explicitly enforced. However, once the value objective increases Q over the execution region, this smoothness is no longer guaranteed. Continuity must therefore be reintroduced as an explicit constraint in the objective. The optimization consequently searches only over $\mathcal{E}$, transferring the requirement of smoothness back to the policy. The formulation above is defined with respect to the timing model in §3.1; therefore, transitions must be collected under the same execution semantics. The asynchronous loop runs during training rollouts rather than only at deployment, ensuring that the replay buffer records the actions actually executed by the robot. In contrast, transitions collected under synchronous execution would contain no explicit region boundaries for the objective to align with.

Exploration is fundamental to reinforcement learning. However, exploration on a real robot is costly: failed trials consume wall-clock time and, for contact-rich tasks, may incur hardware wear or damage. Consequently, human intervention during impending policy failures has become a standard component of real-robot RL rather than merely a convenience [7, 9, 28, 30]. The objective above naturally accommodates such interventions without modification. The critic takes a state and an action chunk as input, and the target in Eq. (3) is independent of the controller that generated the chunk; thus, intervened episodes can be stored in the replay buffer as ordinary transitions. The state augmentation introduced above remains valid under intervention: $\tilde{a}_{[0,n]}$ is defined by the actions actually issued rather than the actions proposed by the policy, and therefore directly records human commands when intervention occurs. Interpreting human actions as value-estimated actions requires no additional transformation in our formulation, unlike settings where the RL variable is represented in a latent action space (§2.4). §3.4 describes the corresponding intervention mechanism.

Under asynchronous deployment, two actions must be distinguished: the action emitted by the policy and the action executed by the robot. Optimizing Q with respect to the former improves behavior only when the two coincide. Furthermore, the action at which Q is evaluated must admit a valid gradient path back to θ . We refer to these two requirements as *agreement* and *differentiability*, respectively. Issuing the policy output directly satisfies both by construction, which is the setting assumed in Eq. (4); this is precisely where the two boundary-smoothing families discussed in §2.1 diverge. A consistency constraint modifies the chunk before execution, ensuring that the policy output and the executed action remain identical. In contrast, blending is performed only after both the new and previous chunks become available. To preserve the agreement requirement, the critic must evaluate the blended action actually executed by the robot. The truncation principle introduced above - that gradients propagate only through actions that enter the environment - extends naturally to this setting. The value gradient should be computed with respect to every chunk contributing to the executed window, restricted to the segment that the chunk contributes. Because these segments are disjoint across transitions, each

chunk is differentiated exactly once. This extension requires an objective horizon exceeding $2n$, an additional actor forward pass for each contributing chunk to recover its output under the current policy parameters, and a reformulation of the continuity constraints around the mixing operator (§5). For simplicity, we adopt direct policy output execution and enforce smoothness through constrained optimization.

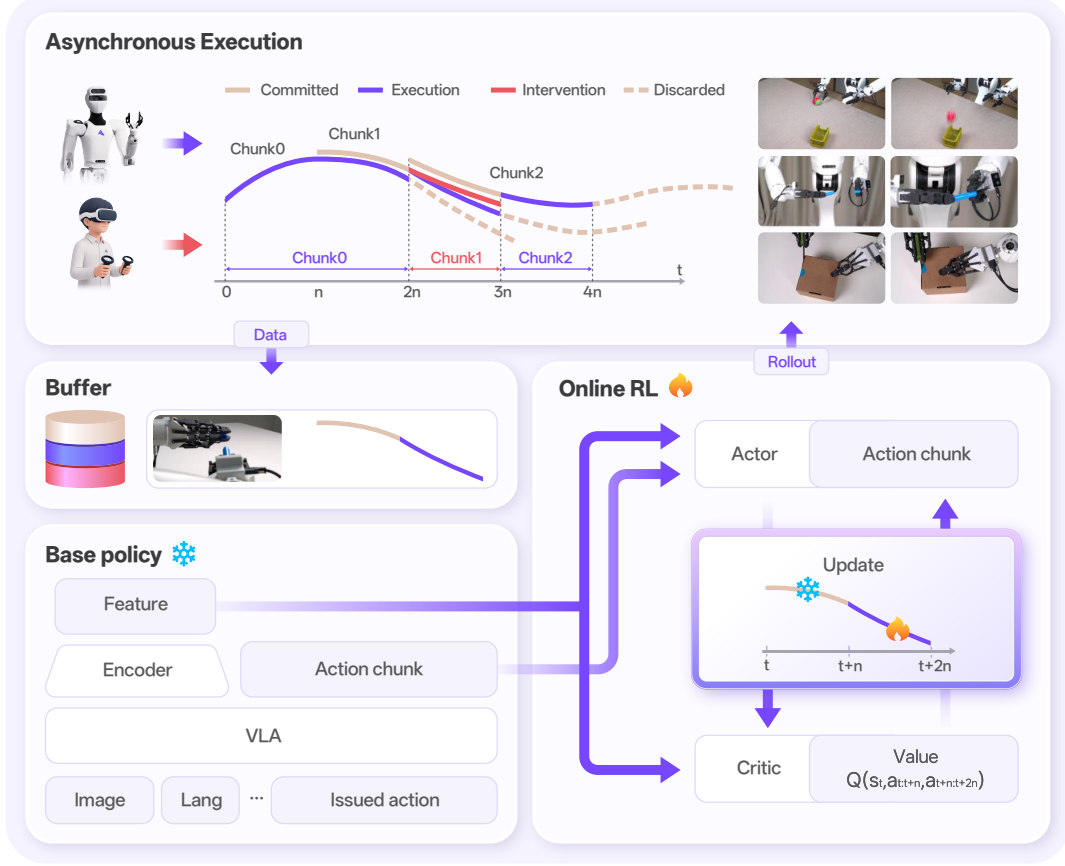
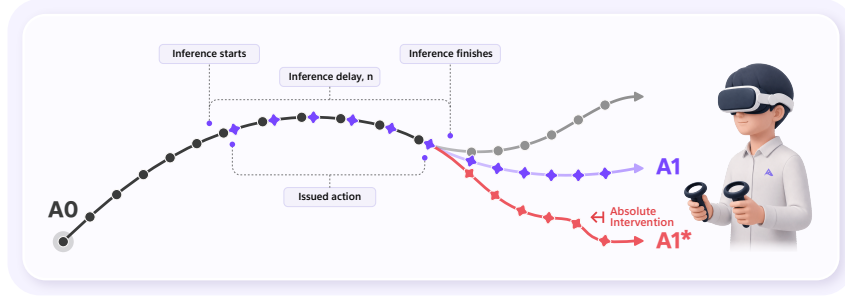


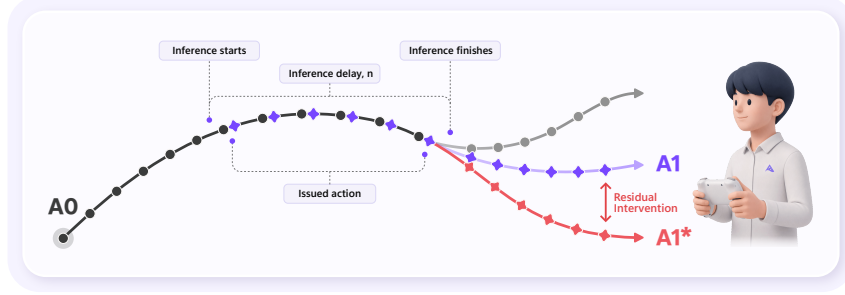
Figure 3 | One instantiation of our asynchronous online RL framework. The frozen base policy produces a latent representation z_t and a reference action chunk $\bar{a}_{t:t+2n}$. A trainable attached network takes $[z_t, s_t, \bar{a}_{t:t+2n}]$ as input and predicts a bounded correction, which is added to the reference action to produce the final action chunk. The critic is conditioned on the corrected action chunk, while gradients are propagated to the actor only through the execution region (§3.3). Human intervention is applied to the issued action in the same raw action space, allowing intervened trajectories to be incorporated directly into training (§3.4).

3.4. Instantiation

We provide a concrete instantiation of the proposed asynchronous online RL framework, as illustrated in Figure 3. Our implementation follows the RLT [9] skeleton: a frozen base policy generates reference action chunks, while a lightweight TD3-style actor-critic accesses the base policy’s internal representations through an RL token and predicts a residual action correction. The following paragraphs clarify what this skeleton provides and where our implementation departs from it. The skeleton requires no structural modification to satisfy the requirements of §3.3 : it provides a chunk-level actor, making frame-indexed gradient truncation expressible; operates in the raw action space, eliminating the need to invert human interventions; and exposes a critic input to which the committed region can be appended as a stop-gradient term.



(a) Absolute intervention.



(b) Residual intervention.

Figure 4 | Two human intervention modes during policy deployment. Under a, the teleoperated action chunk is issued directly and the actor output is discarded. Under b, the human input is added to the actor output, allowing the operator to modify the policy-generated chunk. Both modes are treated identically downstream: the action actually issued is recorded in $\tilde{a}_{[0,n]}$ and used as the BC target.

Keeping the base policy frozen and confining all trainable parameters to a lightweight attached network also makes each gradient step inexpensive relative to a base-policy forward pass. This is important because the update rate is bounded by rollout throughput, while the small number of trainable parameters makes online optimization practical at the data volume generated by a real robot. The choices that follow are therefore implementation details rather than requirements of the framework: §3.1–§3.3 are formulated in terms of generic π and Q , and none of their arguments depends on this particular instantiation.

Human intervention. To evolve the policy during deployment, we support two human intervention modes, as illustrated in Figure 4: **absolute intervention** and **residual intervention**. Intervention is activated and deactivated by a switch on the teleoperation device. The training loop continues while the human is in control: both the base policy and the attached network continue to perform inference, while only the issued action is modified. Under absolute intervention, the teleoperated chunk is issued directly and the actor output is discarded. Under residual intervention, the human input is added to the actor output, allowing the operator to modify the policy-generated chunk rather than replace it. Each mode is supported by a suitable input device: absolute intervention uses VR teleoperation, where the operator’s hand pose directly specifies the target action, while residual intervention uses a hand controller, where stick displacements specify action deltas.

The two intervention modes are suited to different task regimes. Residual intervention preserves the temporal structure of the underlying action chunk while modifying its trajectory, allowing the operator to correct the policy without disrupting its velocity profile. We therefore use it

for dynamic tasks. Absolute intervention, in contrast, gives the operator full control over the action chunk and is therefore suited to tasks requiring greater flexibility and high precision, where the demonstrated motion may differ substantially from the trajectory proposed by the policy. Despite their different interaction mechanisms, the two modes are treated identically downstream. The action actually issued by the operator is recorded in $\tilde{a}_{[0,n)}$ and used as the BC target. Chunks are flagged as intervened at the chunk level, and this flag determines the BC target for the execution region: $a_{[n,2n)}^{\text{target}}$ is the action chunk actually issued for an intervened chunk and the base-policy reference otherwise, while the committed region always uses the reference action as its target. The input to Q is independent of the intervention flag: because the committed region records the action actually issued, it contains the human actions whenever the preceding chunk was intervened. Thus, the state augmentation introduced in §3.3 continues to faithfully represent the action actually in flight.

The base-policy interface. The skeleton’s interface is preserved: the trainable networks access the base policy through a compressed token of its internal representation, while the actor is conditioned on the reference action chunk generated by the base policy. Let π_{base} denote the frozen base policy and E the encoder that reads its internal representation during chunk inference and compresses it into the RL token $z_t = E(\pi_{\text{base}})$. The base policy runs under a TT-RTC scheduler [2] to maintain the timing guarantees of §3.1. Each base-policy forward pass therefore produces both the reference action chunk $\bar{a}_{t:t+2n} = \pi_{\text{base}}(s_t)$ and the RL token z_t on which the trainable networks are conditioned. The committed region provided to Q remains $\tilde{a}_{[0,n)}$ as defined in §3.3, which coincides with the reference chunk only in the absence of intervention. For the concrete instantiation, the state argument of the generic π and Q is represented by the pair (z_t, s_t) . Consequently, each transition must store this pair for the bootstrap state so that the actor target at s_{t+2n} can be evaluated, together with the committed region of that next state, which becomes available only one decision step later during rollout. Algorithm 1 summarizes the complete training loop described in the following two paragraphs: an asynchronous rollout process that generates transitions and an off-policy optimization process that consumes them, with both operating concurrently over a shared replay buffer.

The critic. The critic is an MLP with LayerNorm [57] applied to every hidden layer. It takes $[z_t, s_t, \tilde{a}_{[0,n)}, a_{[n,2n)}]$ as input and predicts a scalar value. The target construction follows TD3 [58]: the target actor’s action chunk is perturbed with clipped Gaussian noise, followed by a pessimistic estimate over the resulting target values. Three aspects, however, depart from the original skeleton. First, the action chunk is explicitly partitioned into the committed and execution regions, with both provided to the critic as inputs. Second, the backup follows the chunk-skipping formulation rather than a single-decision backup. Third, the fixed pair of Q networks is expanded into a REDQ-style [56] ensemble of N independent critics, with the minimum taken over a randomly sampled subset of target critics rather than over the full pair.

The actor. The actor is an MLP of the same architecture. It takes $[z_t, s_t, \bar{a}_{t:t+2n}]$ as input and outputs an action chunk $a_{t:t+2n}$ of length $2n$, as illustrated in Figure 3. In the residual mode used in this work, the network predicts a bounded correction to the reference action, which is added to the reference chunk to produce the final action chunk. The actor loss consists of three terms, defined below for the actor output $a = \pi_{\theta}(z, s, \bar{a})$. The first two losses follow the RLT skeleton: a Q -maximization term regularized by a BC anchor in the spirit of TD3+BC [59], where the anchor

is defined against the reference action rather than a dataset action. Specifically, these correspond to the Q term introduced in §3.3 and a per-frame MSE loss against a target action a^{target} . The third term is specific to our framework: a smoothness regularizer that instantiates the executable set $\mathcal{E}$ introduced in §3.3. We characterize $\mathcal{E}$ by per-frame bounds on velocity, acceleration, and jerk, and relax the corresponding membership constraint into a penalty with overall weight w_{smooth} and relative weight w_k for each derivative order:

$$\mathcal{L}_{\text{actor}} = -Q(z, s, \text{sg}[\tilde{a}_{[0,n]}], a_{[n,2n]}) + w_{\text{bc}} \cdot \|a - a^{\text{target}}\|_{[0,2n]}^2 + w_{\text{smooth}} \sum_{k=1}^3 w_k \cdot \|\Delta^k a\|_{[0,2n]}^2, \quad (5)$$

where $\text{sg}[\cdot]$ denotes the stop-gradient operator: the full action chunk is provided to Q , while gradients with respect to θ propagate only through the execution region.

The replay buffer. The replay buffer is initialized by rolling out the base policy alone under the deployment loop of §3.1, with the attached network inactive; subsequent transitions collected by the asynchronous rollout are appended to the same buffer. This initialization ensures that every transition is collected under the timing assumptions of the objective, rather than from teleoperated demonstrations, and provides the critic with an initial fit to the behavior from which policy improvement begins. The reward r introduced in §3.2 is sparse: it is assigned only at the end of each episode, with success determined by an operator observing the rollout.

The update schedule. Updates are paced by rollout progress rather than wall-clock time. Process 2 of Algorithm 1 performs G critic iterations for every trajectory appended by the rollout, yielding an update-to-data ratio that keeps the update rate bounded by rollout throughput, regardless of the choice of G . The actor and target networks are updated less frequently: one actor update and one soft target-network update are performed every D critic iterations.

4. Experiments

This section evaluates the SmoothRL framework on real-robot manipulation tasks. We consider three tasks spanning two complementary regimes - high-speed dynamic manipulation and high-precision bimanual manipulation - to assess the framework under the competing demands of real-time responsiveness and execution reliability. We use $\pi_{0.5}$ [41], fine-tuned for each task, as the frozen base policy. For each task, we compare its success rate before and after online RL fine-tuning within the SmoothRL framework.

4.1. Platform

All three tasks are performed on Atribot S1, a mobile bimanual robot with 25 degrees of freedom, comprising two 7-DoF arms with parallel grippers, a 4-DoF torso, a 2-DoF head, and a 3-DoF wheeled base. For the base policy, the observation consists of three camera streams (head, left wrist, and right wrist), each processed at 224×224 resolution, together with the robot’s proprioceptive joint state. The base policy issues a 31-dimensional action at 30 Hz, comprising 9-dimensional Cartesian delta poses for the left arm, right arm, and torso end-effector, along

Algorithm 1 SmoothRL training in the asynchronous inference loop

Require: frozen π_{base} with RL-token encoder E , budget n , discount γ , soft-update rate τ , step size η , update-to-data ratio G , actor delay D

Initialize: π_θ and $\{Q_{\phi_i}\}_{i=1}^N$
targets $\theta^- \leftarrow \theta$ and $\phi_i^- \leftarrow \phi_i$
replay buffer $\mathcal{D}$
 $\tilde{a}_{[0,n)} \leftarrow$ the hold action

▸ the robot is stationary until the first chunk arrives

Processes 1 and 2 run concurrently, sharing $\mathcal{D}$ and the parameters.

1: Process 1: asynchronous rollout

2: for $t = 0, n, 2n, \dots$ **do**

3: observe s_t

4: $\bar{a}_{t:t+2n} \leftarrow \pi_{\text{base}}(s_t, \tilde{a}_{[0,n)}); z_t \leftarrow E(\pi_{\text{base}})$

5: $a_{t:t+2n} \leftarrow \pi_\theta(z_t, s_t, \bar{a}_{t:t+2n})$

6: $a_{[0,n)} \leftarrow \tilde{a}_{[0,n)}$ ▸ committed region over $[t, t+n)$

7: $a_{[0,2n)}^{\text{target}} \leftarrow \bar{a}_{[0,2n)}$

8: **if** teleoperation **then**

9: $a_{[n,2n)} \leftarrow a_{[n,2n)}^{\text{human}}$ ▸ absolute or residual, §3.4

10: $a_{[n,2n)}^{\text{target}} \leftarrow a_{[n,2n)}$

11: issue $a_{[n,2n)}$

12: record a transition $\tau_t = (s_t, z_t, \tilde{a}_{[0,n)}, a_{[n,2n)}, a_{[0,2n)}^{\text{target}})$ ▸ finished at $t+2n$

13: **if** $t \geq 2n$ **then**

14: $\tau_{t-2n} \leftarrow \tau_{t-2n} \cup (r_{t-2n}, s_t, z_t, \bar{a}_{t:t+2n}, a_{[0,n)})$ ▸ reward over $[t-2n, t)$; next state

15: append τ_{t-2n} to $\mathcal{D}$

16: $\tilde{a}_{[0,n)} \leftarrow a_{[n,2n)}$ ▸ update committed region for next step

17: Process 2: off-policy updates

18: for each trajectory appended by Process 1 **do**

19: **for** G iterations **do**

20: sample a batch $B \subset \mathcal{D}$

21: **for** each $\tau = (s, z, \tilde{a}_{[0,n)}, a_{[n,2n)}, a_{[0,2n)}^{\text{target}}, r, s', z', \bar{a}', \tilde{a}'_{[0,n)}) \in B$:

22: draw $\mathcal{M} \subset \{1, \dots, N\}$ at random

23: $a' \leftarrow \pi_{\theta^-}(z', s', \bar{a}') + \epsilon$ ▸ ϵ clipped Gaussian

24: $y \leftarrow r + \gamma^{2n} \min_{i \in \mathcal{M}} Q_{\phi_i^-}(z', s', \tilde{a}'_{[0,n)}, a'_{[n,2n)})$

25: $\phi_i \leftarrow \phi_i - \eta \nabla_{\phi_i} \sum_B (Q_{\phi_i}(z, s, \tilde{a}_{[0,n)}, a_{[n,2n)}) - y)^2$ for all i

26: **if** every D -th iteration **then**

27: $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}_{\text{actor}}$ ▸ Eq. (5)

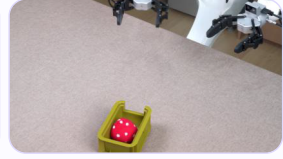
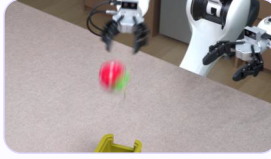
28: $\theta^- \leftarrow \tau\theta + (1-\tau)\theta^-$

29: $\phi_i^- \leftarrow \tau\phi_i + (1-\tau)\phi_i^-$ for all i

with 2 gripper commands and 2 head-joint commands. The attached actor and critic modify only the 20 arm-action dimensions, leaving the torso and head actions unchanged. The control frequency determines the frame rate at which the latency budget n in §3.1 is measured. Inference is requested at 5 Hz, so a new action chunk arrives every 200 ms, corresponding to a latency budget of $n = 6$ frames. Each chunk therefore contributes 6 frames to its execution region while the subsequent chunk is being inferred. The base policy is trained to predict $H = 32$ frames



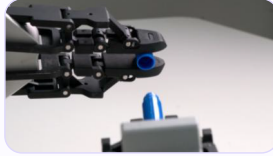
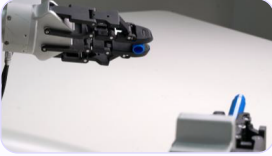
Dynamic Tossing:



Grasp an object and toss it into a target container.



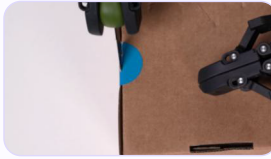
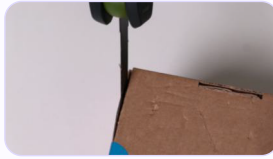
Pen Capping:



Align the pen with its cap, secure the cap onto the pen, and release the cap.



Box Opening:



Stabilize the box with the left hand while the right hand precisely aligns with the seam, cuts the tape, and opens the flaps.

Figure 5 | The three real-robot tasks evaluated in this work: dynamic object tossing into a target bin, bimanual pen capping, and high-precision box opening by slitting the sealing tape and opening the flaps.

per chunk; thus, the committed and execution regions occupy 12 frames in total, while the remaining 20 frames form the discarded region and are superseded by the subsequent chunk before execution.

4.2. Task Specification

Dynamic tossing. The robot is instructed to grasp a randomly placed object and toss it into a randomly placed bin. Successful tossing requires the end effector to accumulate sufficient velocity throughout the swing prior to release, rather than attaining the target velocity at a single instant. This makes the task particularly sensitive to asynchronous execution: a pause at a chunk boundary can bring the arm nearly to rest, leaving insufficient time for the remaining swing to recover the required release velocity. Consequently, successful execution requires both precise release timing and continuous motion across chunk boundaries. The bin opening is approximately 7×12 cm, while the object is approximately 6×7 cm, making the task challenging in both throwing speed and spatial accuracy.

Pen capping. The robot is instructed to align the pen with its cap, secure the cap onto the pen, and release the cap. Successful insertion requires the relative pose between the pen and cap to fall within a clearance tolerance of approximately 5 mm. Errors beyond this tolerance cannot be reliably corrected by applying additional force, as the cap tends to deflect rather than seat properly. Since both arms are actively controlled, the insertion accuracy depends on their relative pose rather than the motion of either arm alone. This makes the task particularly challenging for the pretrained base policy: although it typically reaches the vicinity of the desired configuration, small residual pose errors can prevent successful insertion. The attached network is therefore responsible for correcting these residual errors and achieving the required insertion precision.

Box opening. The task requires the robot to stabilize a package box with its left hand while the right arm inserts a blade into the seam between the flaps and draws it along the seam to cut the tape, after which the flaps are opened. The blade is approximately 1 mm wide, while the seam is only 2–3 mm wide, requiring the insertion to locate the seam with approximately millimeter-level precision. An insertion error can fail in either direction: the blade may cut into the cardboard and damage the box, or it may miss the tape and leave the box sealed. Once the blade is inserted, the seam guides its motion, substantially reducing the positional demands during the subsequent cutting phase; the blade only needs to maintain sufficient depth to sever the tape. Thus, the primary challenge lies in precise insertion, where the pretrained policy reaches the vicinity of the desired configuration but remains insufficiently reliable.

4.3. Training Setup

Hyperparameters. The attached actor and critic are implemented as 3-layer MLPs with 512 hidden units per layer. We use an update-to-data ratio of $G = 5$ and an actor update delay of $D = 5$, such that five gradient updates are performed for each newly collected trajectory, while the actor and target networks are updated every fifth gradient step. The training batch size is set to 256. We set the correction bound to 0.05, reflecting that the pretrained base policy is already close to successful behavior and therefore requires only small corrective adjustments. All remaining hyperparameters follow standard off-policy actor-critic practice and are kept fixed across all tasks.

Reward labeling and human intervention. The sparse reward defined in §3.4 is provided by an operator through a gamepad or VR interface. An episode is terminated and assigned a binary reward only when the operator judges the task to be complete; no intermediate rewards are provided at any earlier timestep. Following each episode, the robot is returned to its initial pose and the scene is reset for the next trial. Evaluation is based solely on task success rate. During training, the operator intervenes as needed to maintain a balanced ratio of successful and failed episodes. Human-intervened actions are incorporated through the raw action space, such that they are stored and trained on in the same manner as policy-generated actions.

The two intervention modes introduced in §3.4 exhibit markedly different performance on the dynamic tossing task, motivating the use of residual intervention. This difference is most pronounced for bin positions farthest from the robot, which require higher release velocities and are therefore more challenging. For these configurations, direct chunk-level teleoperation via VR achieves a success rate of approximately 30%, compared with approximately 80% when the operator provides a residual correction to the base policy’s action chunk. Direct teleoperation

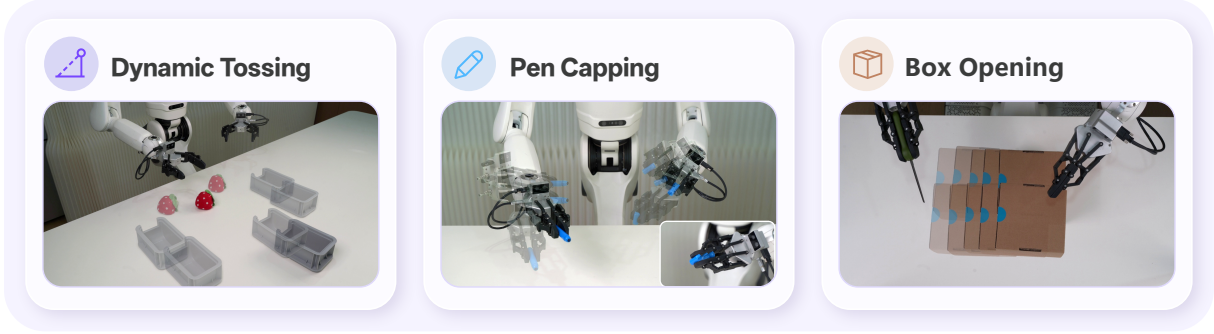


Figure 6 | Initial-state configurations used for evaluation. Ghosted overlays indicate the sampled configurations. The inset illustrates the variation in the in-hand grasp pose, which is manually set during reset and is not included in the enumerated configuration combinations.

requires the operator to control both the direction and magnitude of the release velocity while simultaneously maintaining continuity when transitioning between human and policy control. Residual intervention instead preserves the base policy’s velocity profile and requires the operator only to compensate for its deviations. This substantially reduces the control burden while better preserving motion continuity across intervention boundaries.

4.4. Evaluation

Protocol. Each task is evaluated across a predefined set of initial-state configurations. Box opening uses 10 box positions, dynamic tossing uses 18 configurations comprising 3 object positions $\times$ 6 target-bin positions, and pen capping uses 12 configurations comprising 4 initial hand states $\times$ 3 pen-cap positions. Figure 6 visualizes the configurations for each task, which are identical to those used during online fine-tuning. Each configuration is evaluated once, yielding 10, 18, and 12 episodes per checkpoint for box opening, dynamic tossing, and pen capping, respectively. An episode terminates when the operator judges either that the task has been successfully completed or that it can no longer be completed. An episode is counted as successful only in the former case, and the success rate is computed as the number of successful episodes divided by the total number of evaluation episodes.

Comparison. Each task is evaluated at four points throughout the online RL process. The first is the frozen base policy $\pi_{0.5}$ [41], pretrained on the task and deployed with its parameters fixed, without the attached network in the control loop. This baseline represents the performance attainable through supervised pretraining alone and serves as the zero point on the horizontal axis of the learning curves shown in Figure 7. Supervised pretraining uses 1,500 teleoperated demonstrations for box opening and pen capping, and 500 for dynamic tossing. The latter requires fewer demonstrations because it is substantially more difficult to collect: across all bin positions, teleoperation succeeds on only approximately 50% of attempts, as human operators face the same challenge of maintaining a continuous swing. Online RL starts from the corresponding pretrained policy. We first collect 50 rollout episodes under the frozen base policy alone to seed the replay buffer. The attached actor and critic are then introduced into the loop, and subsequent rollout episodes are collected under the online RL procedure. The remaining three evaluation points are checkpoints from this same run, taken after 150, 200, and 250 rollout episodes. All four checkpoints are evaluated on the same hardware using the same asynchronous inference loop and latency budget. Thus, the only variable across evaluation

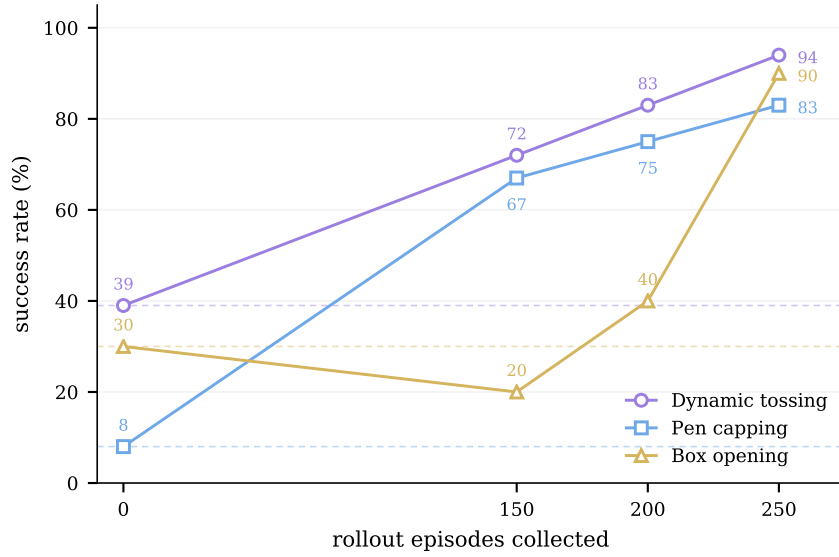


Figure 7 | Success rate versus online interaction episodes, with one curve per task. The zero marker denotes the frozen base policy, with its success rate extended as a dashed line for reference. Each point reports success over N evaluation episodes at the corresponding checkpoint as specified in §4. The three nonzero checkpoints come from a single RL run per task; lines are connected for visualization only.

points is the amount of online fine-tuning applied to the attached network.

4.5. Results

Figure 7 and Table 1 report the success rates of all three tasks as a function of the number of rollout episodes collected during online RL. After 250 episodes, all three tasks substantially outperform the frozen base policy: dynamic tossing improves from 39% to 94%, pen capping from 8% to 83%, and box opening from 30% to 90%. The following paragraphs examine what the frozen base policy fails to capture on each task, how much of this gap is recovered through online fine-tuning, and the failure modes that remain.

Failure modes of the base policy. Across all three tasks, failures of the frozen base policy are systematic rather than random, primarily arising from insufficient adaptation to variations in object or target poses. In pen capping, the policy produces similar motion trajectories across different cap positions and fails to adequately adjust to the varying cap poses, resulting in only an 8% success rate. In box opening, the blade exhibits a consistent leftward bias, causing it to pierce the cardboard rather than enter the tape seam. In dynamic tossing, the policy fails to appropriately modulate the release velocity with target distance: it tends to overshoot nearby bins and undershoot distant ones, while also exhibiting systematic left-right deviations for targets on opposite sides.

What online RL remedies. The attached network progressively corrects the systematic offsets of the frozen base policy, with the final checkpoint substantially improving performance across all three tasks, although the learning dynamics vary by task. Dynamic tossing and pen capping improve monotonically across checkpoints, with the largest gains occurring within the first

Task	Number of Rollout Episodes	Success Rate
Dynamic Tossing	0 (base policy)	39%
	150	72%
	200	83%
	250	94%
Pen Capping	0 (base policy)	8%
	150	67%
	200	75%
	250	83%
Box Opening	0 (base policy)	30%
	150	20%
	200	40%
	250	90%

Table 1 | Success rate at each checkpoint, with the cumulative number of rollout episodes collected before evaluation. Each success rate is computed from one episode per initial-state configuration under the evaluation protocol specified in §4.4.

150 episodes. For pen capping in particular, the success rate increases from 8% to 67% during this initial phase, indicating that the dominant systematic offset is largely corrected early, after which performance is primarily limited by residual variation around the corrected trajectory. Box opening exhibits a different learning trajectory: performance initially drops to 20% after 150 episodes, falling below the frozen base policy, before recovering to 40% and ultimately reaching 90%. This transient degradation likely reflects the exploration required for the attached network to discover effective corrections while adapting to the task.

Closing the last millimeter. Figure 8 shows representative failures on each task, at the final checkpoint (250 episodes) and under the frozen base policy for comparison. Comparing the two rows of Figure 8 shows the failures moving closer to success: the base policy misses in a consistent direction and by a visible margin, whereas what remains after RL is a small spread around the correct pose. On dynamic tossing, the object lands closer to the bin and breaks the base policy’s pattern of fixed bias; on box opening, the blade’s deviation from the seam is much smaller, with left-right deviations of 1–2 mm around the correct position; on pen capping, the axial misalignment between cap and barrel is likewise much smaller, a small spread around the correct pose. These deviation magnitudes are precisely what makes these tasks challenging: success is not about reaching the target region but closing the last few millimeters, and an error small enough to be barely distinguishable in the image is sufficient to fail the episode.

5. Conclusion, Limitations and Future Work

This work presents SmoothRL, a framework for fine-tuning pretrained robot policies with online RL under asynchronous inference, improving success on fine-grained real-robot tasks while preserving real-time responsiveness. Asynchrony breaks the correspondence between the action optimized by the value gradient and the action actually executed by the environment. Our chunk role partitioning and gradient truncation explicitly restore this correspondence by restricting policy updates to the executed region. Operating in the raw action space further enables direct

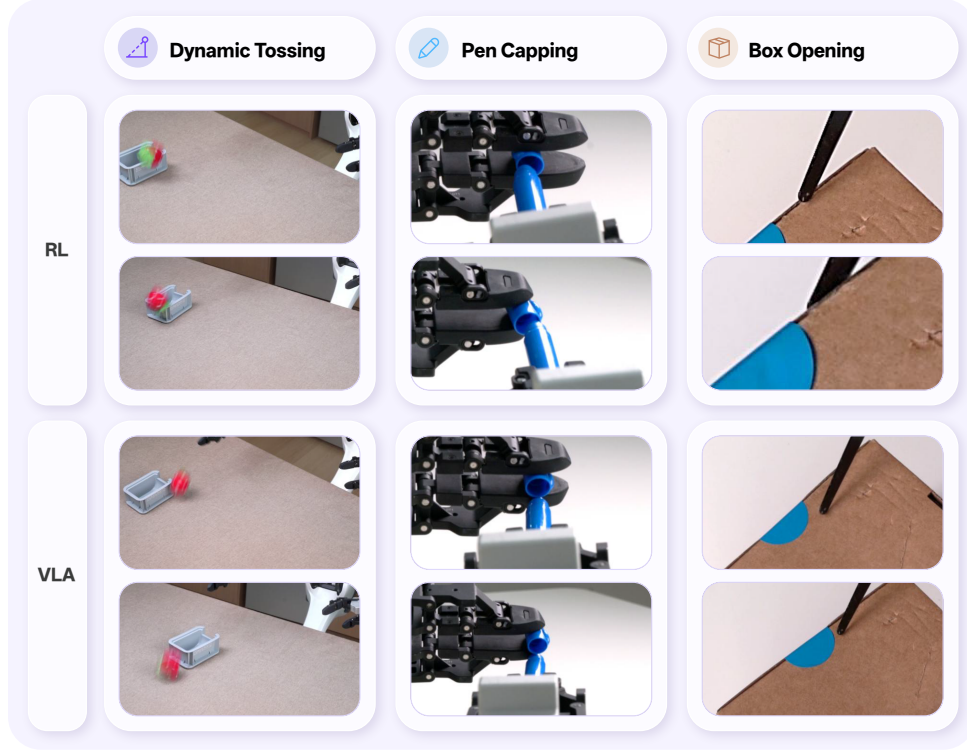


Figure 8 | Representative failures on dynamic tossing, pen capping, and box opening. Columns denote tasks; top and bottom rows show failures of the RL policy and frozen base policy, respectively. RL-tuned failures are near misses, while the base policy exhibits larger, systematic deviations.

state augmentation for concurrent execution and allows human intervention to be incorporated naturally as training data.

Limitations

The proposed online RL framework with asynchronous inference is general, yet the current instantiation has two known limitations:

1. **Inference latency constraint.** The proposed framework operates under a fixed inference frequency, requiring each chunk-level inference to complete within a predefined latency budget. When temporal fluctuations in inference latency cause the actual computation time to exceed this budget, scheduled chunk handover cannot be guaranteed. The resultant timing misalignment destabilizes the asynchronous execution loop.
2. **Output expressiveness bounded by the base policy.** The expressive capacity of the RL module is constrained by the base policy in two key aspects. First, the conditional features used for RL optimization are derived from the internal representations of the VLA model, so the perceptual capability of the RL branch is upper-bounded by the information richness of VLA embeddings. Second, in the current implementation, the RL module produces only bounded residual corrections upon the reference actions provided by the base policy. The correction magnitude is confined within a fixed local neighborhood, whose radius balances the effective correction range and the strength of trust-region regularization. When the base policy exhibits systematic biases on target-domain data, such local residual corrections lack sufficient expressiveness to rectify erroneous policy behaviors.

Future Work

Three broader directions emerge from this study. **First, the proposed framework may extend beyond the particular policy parameterization considered here.** Since §3.3 constrains only where the value gradient is applied, the same principle could accommodate a wider class of offline-to-online learning algorithms and substantially different policy architectures. In particular, the value gradient need not be restricted to a small attached network; it could propagate through the parameters of an entire generative policy, as in QAM [55], whose adjoint terminal condition $\nabla_a Q$ over the generated chunk provides a natural mechanism for expressing the truncation induced by asynchronous execution. Such formulations, however, would need to treat the committed prefix explicitly: preserving previously issued actions becomes a prefix-constrained generation problem rather than an output-space overwrite. This perspective points toward a more general formulation of online policy optimization under partially committed generation. **Second, the interaction between policy learning and action blending warrants further investigation.** While the framework rules out an external mixing operator, it does not preclude incorporating the mixture directly into the learning objective. Doing so would allow the critic to evaluate the actions that are ultimately executed, potentially providing a more faithful training signal under asynchronous deployment. This comes at the cost of a broader temporal dependency in the objective, additional continuity constraints around the blending operator, and an additional actor forward pass for each contributing chunk. More fundamentally, the blending weight typically decays toward the junction between chunks, precisely where continuity is most critical, suggesting a potentially unfavorable trade-off between execution smoothness and the strength of the learning signal. Understanding this trade-off could lead to principled formulations that jointly optimize policy improvement and temporal consistency. **Third, scaling the evaluation to substantially broader and more diverse task distributions is essential for characterizing the fundamental expressiveness limits of the proposed adaptation mechanism.** In particular, systematically increasing the distributional shift from the pretrained base policy could reveal when local correction around the reference action ceases to be sufficient, and whether this limitation is best addressed through a more expressive adapter, a fully generative policy update, or a mechanism that can progressively relax the reference constraint. Such experiments would help distinguish limitations arising from the learning algorithm itself from those imposed by the chosen policy parameterization.

6. Contributions

- **Contributors:** Guang Gao*, Yuxuan Nong*, Baifu Huang
- **Project Lead:** Jianan Wang

* Equal contribution.

References

- [1] Kevin Black et al. Real-Time Execution of Action Chunking Flow Policies. arXiv:2506.07339, 2025.
- [2] Kevin Black et al. Training-Time Action Conditioning for Efficient Real-Time Chunking. arXiv:2512.05964, 2025.
- [3] Yufeng Liu et al. Learning Native Continuation for Action Chunking Flow Policies. arXiv:2602.12978, 2026.
- [4] Dongyang Liu et al. Action-Prior Denoising for Smooth Real-Time Chunking. arXiv:2605.25537, 2026.
- [5] Cheng Chi et al. Diffusion Policy: Visuomotor Policy Learning via Action Diffusion. arXiv:2303.04137, 2023.
- [6] Tony Z. Zhao et al. Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware. arXiv:2304.13705, 2023.
- [7] Perry Dong et al. EXPO-FT: Sample-Efficient Reinforcement Learning Finetuning for Vision-Language-Action Models. arXiv:2605.25477, 2026.
- [8] Chenyu Yang et al. SERNF: Sample-Efficient Real-World Dexterous Policy Fine-Tuning via Action-Chunked Critics and Normalizing Flows. arXiv:2602.09580, 2026.
- [9] Charles Xu et al. RL Token: Bootstrapping Online RL with Vision-Language-Action Models. arXiv:2604.23073, 2026.
- [10] Dongchi Huang et al. CO-RFT: Efficient Fine-Tuning of Vision-Language-Action Models through Chunked Offline Reinforcement Learning. arXiv:2508.02219, 2025.
- [11] Wenli Xiao et al. Self-Improving Vision-Language-Action Models with Data Generation via Residual RL. arXiv:2511.00091, 2025.
- [12] Lars Ankile et al. Residual Off-Policy RL for Finetuning Behavior Cloning Policies. arXiv:2509.19301, 2025.
- [13] Guozheng Ma et al. What Makes Value Learning Efficient in Residual Reinforcement Learning? arXiv:2602.10539, 2026.
- [14] Qiyang Li et al. Decoupled Q-Chunking. arXiv:2512.10926, 2025.
- [15] Guanxing Lu et al. VLA-RL: Towards Masterful and General Robotic Manipulation with Scalable Reinforcement Learning. arXiv:2505.18719, 2025.
- [16] Haozhan Li et al. SimpleVLA-RL: Scaling VLA Training via Reinforcement Learning. arXiv:2509.09674, 2025.
- [17] Yanjiang Guo et al. Improving Vision-Language-Action Model with Online Reinforcement Learning. arXiv:2501.16664, 2025.
- [18] Si-Cheng Wang et al. VLA Model Post-Training via Action-Chunked PPO and Self Behavior Cloning. arXiv:2509.25718, 2025.

- [19] Kang Chen et al. π_{RL} : Online RL Fine-tuning for Flow-based Vision-Language-Action Models. arXiv:2510.25889, 2025.
- [20] Siting Wang et al. π -StepNFT: Wider Space Needs Finer Steps in Online RL for Flow-based VLAs. arXiv:2603.02083, 2026.
- [21] Allen Z. Ren et al. Diffusion Policy Policy Optimization. arXiv:2409.00588, 2024.
- [22] Tonghe Zhang et al. ReinFlow: Fine-tuning Flow Matching Policy with Online Reinforcement Learning. arXiv:2505.22094, 2025.
- [23] Seohong Park et al. Flow Q-Learning. arXiv:2502.02538, 2025.
- [24] Kun Lei et al. RL-100: Performant Robotic Manipulation with Real-World Reinforcement Learning. arXiv:2510.14830, 2025.
- [25] Yuhui Chen et al. ConRFT: A Reinforced Fine-tuning Method for VLA Models via Consistency Policy. arXiv:2502.05450, 2025.
- [26] Yi Wang et al. Learning While Deploying: Fleet-Scale Reinforcement Learning for Generalist Robot Policies. arXiv:2605.00416, 2026.
- [27] Perry Dong et al. EXPO: Stable Reinforcement Learning with Expressive Policies. arXiv:2507.07986, 2025.
- [28] Jianlan Luo et al. Precise and Dexterous Robotic Manipulation via Human-in-the-Loop Reinforcement Learning. arXiv:2410.21845, 2024.
- [29] Checheng Yu et al. χ_0 : Resource-Aware Robust Manipulation via Taming Distributional Inconsistencies. arXiv:2602.09021, 2026.
- [30] Physical Intelligence. $\pi_{0.6}^*$: a VLA That Learns From Experience. arXiv:2511.14759, 2025.
- [31] ByteDance Seed. GR-RL: Going Dexterous and Precise for Long-Horizon Robotic Manipulation. arXiv:2512.01801, 2025.
- [32] Andrew Wagenmaker et al. Steering Your Diffusion Policy with Latent Space Reinforcement Learning. arXiv:2506.15799, 2025.
- [33] Qiyang Li et al. Reinforcement Learning with Action Chunking. arXiv:2507.07969, 2025.
- [34] Ge Li et al. TOP-ERL: Transformer-based Off-Policy Episodic Reinforcement Learning. arXiv:2410.09536, 2024.
- [35] Ted Xiao et al. Thinking While Moving: Deep Reinforcement Learning with Concurrent Control. arXiv:2004.06089, 2020.
- [36] Ayoub Agouzoul. Understanding Asynchronous Inference Methods for Vision-Language-Action Models. arXiv:2605.08168, 2026.
- [37] Jiaming Tang et al. VLASH: Real-Time VLAs via Future-State-Aware Asynchronous Inference. arXiv:2512.01031, 2025.
- [38] Atribot Team. Asynchronous Fast-Slow Vision-Language-Action Policies for Whole-Body Robotic Manipulation. arXiv:2512.20188, 2025.

- [39] Kohei Sendai et al. Leave No Observation Behind: Real-time Correction for VLA Action Chunks. arXiv:2509.23224, 2025.
- [40] Physical Intelligence. π_0 : A Vision-Language-Action Flow Model for General Robot Control. arXiv:2410.24164, 2024.
- [41] Physical Intelligence. $\pi_{0.5}$: a Vision-Language-Action Model with Open-World Generalization. arXiv:2504.16054, 2025.
- [42] Songming Liu et al. RDT-1B: a Diffusion Foundation Model for Bimanual Manipulation. arXiv:2410.07864, 2024.
- [43] Yue Su et al. World Guidance: World Modeling in Condition Space for Action Generation. arXiv:2602.22010, 2026.
- [44] Lin Li et al. Causal World Modeling for Robot Control. arXiv:2601.21998, 2026.
- [45] BeingBeyond Team. Being-H0.7: A Latent World-Action Model from Egocentric Videos. arXiv:2605.00078, 2026.
- [46] Atribot Team. Towards Predictive, Aligned, and Scalable Robot Learning. arXiv:2607.11270, 2026.
- [47] NVIDIA. GR00T N1: An Open Foundation Model for Generalist Humanoid Robots. arXiv:2503.14734, 2025.
- [48] Tianyuan Yuan et al. Fast-WAM: Do World Action Models Need Test-time Future Imagination? arXiv:2603.16666, 2026.
- [49] Moo Jin Kim et al. Cosmos Policy: Fine-Tuning Video Models for Visuomotor Control and Planning. arXiv:2601.16163, 2026.
- [50] Xue Bin Peng et al. Advantage-Weighted Regression: Simple and Scalable Off-Policy Reinforcement Learning. arXiv:1910.00177, 2019.
- [51] Kevin Frans et al. Diffusion Guidance Is a Controllable Policy Improvement Operator. arXiv:2505.23458, 2025.
- [52] Chenxiao Gao et al. FlowRL: A Taxonomy and Modular Framework for Reinforcement Learning with Diffusion Policies. arXiv:2603.27450, 2026.
- [53] Mitsuhiko Nakamoto et al. Steering Your Generalists: Improving Robotic Foundation Models via Value Guidance. arXiv:2410.13816, 2024.
- [54] Jacky Kwok et al. RoboMonkey: Scaling Test-Time Sampling and Verification for Vision-Language-Action Models. arXiv:2506.17811, 2025.
- [55] Qiyang Li, Sergey Levine. Q-learning with Adjoint Matching. arXiv:2601.14234, 2026.
- [56] Xinyue Chen et al. Randomized Ensembled Double Q-Learning: Learning Fast Without a Model. arXiv:2101.05982, 2021.
- [57] Jimmy Lei Ba et al. Layer Normalization. arXiv:1607.06450, 2016.
- [58] Scott Fujimoto et al. Addressing Function Approximation Error in Actor-Critic Methods. arXiv:1802.09477, 2018.

- [59] Scott Fujimoto, Shixiang Shane Gu. A Minimalist Approach to Offline Reinforcement Learning. arXiv:2106.06860, 2021.
- [60] Tuomas Haarnoja et al. Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor. arXiv:1801.01290, 2018.